

Maui Surf House Term Project Study Guide

Project in one minute

Maui Surf House is a full-stack vacation-rental application for Australian surfers visiting Maui. The React frontend displays the property, amenities, tourism charts, and weather. The Express backend provides JSON and browser routes, reads and writes MongoDB through Mongoose, and protects admin actions with local and Google authentication. The app is deployed as two Render services: a backend Web Service and a React Static Site.

Architecture

```
React/Vite frontend
-> fetch with credentials
Express backend
-> Mongoose queries
MongoDB Atlas

Google OAuth
-> Passport callback
-> find/link/create User
-> express-session + connect-mongo
-> connect.sid cookie
```

Weeks 2-3: HTML, CSS, JavaScript

The early assignments established HTML structure, CSS presentation, JavaScript functions, events, random values, and DOM updates. These ideas continue in the final project: React replaces direct DOM manipulation, but the browser still receives HTML, CSS, and JavaScript and responds to user events.

Be ready to explain the DOM, event listeners, functions, state, and why content and presentation are separated.

Weeks 4-6: Node, Express, routes, and APIs

Node runs JavaScript on the server. Express maps HTTP methods and paths to handlers. Middleware runs during the request-response pipeline.

Important final-project middleware includes `express.json()`, `express.urlencoded()`, sessions, Passport initialization, Passport session restoration, CORS headers, and authorization guards.

The basic request flow is:

```
Browser -> Express route -> database or external API -> response -> browser
```

`GET` reads, `POST` creates or performs an action, and `PUT` updates. JSON is the structured format used between the React frontend and Express backend.

Weeks 8-9: External APIs and dashboard data

The dashboard fetches live OpenWeatherMap forecast data using `VITE_WEATHER_KEY`. It handles three states: loading, successful live data, and fallback data when the key is missing or the request fails.

The weather key is exposed to the browser because it begins with ``VITE_``; it is not a true backend secret. It should be restricted and rotated if exposed. Database credentials and session secrets remain backend-only.

Weeks 10-11: MongoDB and Mongoose

``models/Property.js`` defines the source-of-truth property document. It contains the name, island, type, description, amenities, target segment, image URL, and embedded reviews.

Reviews are embedded because they belong to one property and are normally displayed with it. The review schema validates ratings from 1 through 5, requires the guest name and comment, and uses ``{_id: false}`` so each embedded review does not need its own identifier.

``seed.js`` deletes and inserts the known Maui Surf House record and creates local accounts with bcrypt hashes. The seed script is useful for setup but should not be run casually against production data because it deletes records.

Week 11 routes

The important backend routes are:

- ``GET /properties``: returns JSON for API clients or renders the EJS page for browsers.
- ``GET /properties/:id``: returns one property.
- ``POST /properties/:id/reviews``: validates and saves a review.
- ``GET /admin/dashboard``: returns protected admin metrics.
- ``PUT /admin/properties/:id``: updates allowed property fields.

The ``buildPropertyFilter()`` function turns query parameters such as island and rating ranges into a MongoDB filter. The route also clamps ratings to the valid 1-5 range and reports invalid numeric input.

Weeks 12-13: React and Chart.js

``src/main.jsx`` mounts the React application. ``App.jsx`` owns property data, loading state, login state, and the session banner. ``MarketingPage`` renders the public property experience. ``Dashboard`` renders visitor arrivals, hotel occupancy, and weather forecast charts.

``useState()`` stores changing values. ``useEffect()`` runs side effects such as fetch requests after rendering. ``useMemo()`` keeps chart configuration stable until its input data changes. React re-renders when state changes.

Chart.js is modular, so the app registers the elements, scales, legend, tooltip, and line/bar features it uses. The dashboard receives the property through props rather than fetching duplicate property data itself.

Week 14: local authentication

``POST /auth/login`` normalizes the username, finds the user, compares the submitted password with ``bcrypt.compare()``, and calls ``req.login()`` to establish a Passport session. The database stores ``passwordHash``, never the plain password.

Authentication answers "Who is the user?" Authorization answers "What may the user do?" ``requireAuth`` returns 401 when no session exists. ``requireAdmin`` returns 401 when unauthenticated and 403 when an authenticated

user lacks the admin role.

The seeded accounts are `admin / Admin123!` and `visitor / User123!`. Google-created accounts receive the default `user` role.

Week 15: Google OAuth 2.0

The OAuth authorization-code flow is:

1. The user clicks Continue with Google.
2. Express redirects to Google with the client ID, callback URL, scope, and account-selection prompt.
3. Google authenticates the user and redirects to `/auth/google/callback` with a temporary code.
4. Passport exchanges the code server-side for Google profile data.
5. The app finds the user by Google ID, links by verified email, or creates a new user.
6. Passport stores the user ID in the session and Express redirects to the frontend.

The production callback is:

```
https://ics385spring2026-a20r.onrender.com/auth/google/callback
```

The callback belongs to the backend because the client secret must never be placed in browser code. Google Cloud Console and Render must contain the exact same callback URL.

The User model contains `googleId`, email, display name, username, password hash, role, provider, and profile photo. New Google users receive a unique username such as `google\_<google-id>` because the MongoDB username index is unique.

Sessions and cookies

The server uses `express-session` with `connect-mongo`, so the session survives server restarts and stores the session record in MongoDB. Passport serializes the user ID into the session and deserializes it on later requests.

The browser session cookie uses `httpOnly`, `secure` in production, and `SameSite=None` in production because the frontend and backend are separate Render services. React must send `credentials: "include"` on login, logout, and session requests.

The frontend calls `GET /auth/session` after loading. When the response contains `authenticated: true`, React sets `sessionUser` and renders the Signed in as banner.

Logout behavior

Logout is a `POST /auth/logout` action, not a browser-address-bar GET. The server calls `req.logout()`, destroys the session, and clears `connect.sid`. The frontend clears its local user state. Google may still remember the Google account; application logout does not log the user out of Google. The OAuth route uses `prompt: "select\_account"` so the next login can choose an account.

Testing

``tests/auth.test.js`` uses Jest and Supertest. ``request.agent(app)`` preserves cookies between requests so tests can simulate a browser session. The tests verify unauthenticated protection, normal-user authorization failure, admin login, admin property updates, logout, and the Google redirect. The current suite has six passing tests.

Deployment

The backend Render Web Service uses root ``week15/term-project``, build ``npm install``, and start ``npm start``. The frontend Render Static Site uses root ``week15/term-project/react-marketing``, build ``npm install && npm run build``, and publish directory ``dist``.

The frontend must receive:

```
VITE_API_BASE_URL=https://ics385spring2026-a20r.onrender.com
```

The backend must receive ``FRONTEND_ORIGIN`` set to the exact frontend URL and the production Google callback variables. Frontend Vite environment changes require a new frontend build.

Issues found and solved

- Google returned ``redirect_uri_mismatch``: registered the exact Render backend callback in Google Cloud and Render.
- Google callback returned HTTP 500: accepted the Google profile email verification field formats used by the provider.
- MongoDB rejected new Google users with ``username: null``: generated a unique ``google_<id>`` username.
- Frontend session returned false after successful OAuth: enabled production cross-origin cookies with ``SameSite=None; Secure`` and sent fetch credentials.
- Session responses were cached: added ``Cache-Control: no-store`` and ``cache: "no-store"``.
- Logout reused the old session: called ``req.logout()`` before destroying the session and clearing the cookie.
- Google silently reused the same account: added ``prompt: "select_account"``.
- Render frontend used stale output: redeployed with the correct root directory, build command, publish directory, and environment variable.

Professor questions to rehearse

1. Why is the Google callback on the backend?
2. What is the difference between authentication and authorization?
3. Why are reviews embedded in the property document?
4. Why do tests use ``request.agent()``?
5. What do 401 and 403 mean in this app?
6. Why does React use ``credentials: "include"``?
7. Why is ``SameSite=None`` needed for the two Render services?
8. What happens if OpenWeatherMap fails?

9. What does the Passport verify callback do?

10. What would you improve next?

A strong reflection is: the core system works end-to-end, and the next security improvements would be adding Helmet, adding express-validator to input routes, rotating exposed frontend API keys, and expanding tests for review validation and Google account linking.